

SQL Window Functions – Interview Questions

1. What are window functions in SQL? How do they differ from aggregate functions?
2. Explain the difference between GROUP BY and OVER() clause.
3. What is the difference between PARTITION BY and ORDER BY in a window function?
4. What are frames (ROWS BETWEEN / RANGE BETWEEN)? How do they affect calculations?
5. Difference between ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW vs. RANGE BETWEEN?
6. What is the default frame specification if you don't mention ROWS?
7. Explain the difference between RANK(), DENSE_RANK(), and ROW_NUMBER(). Give examples.
8. How does NTILE(n) work? Where can it be applied?
9. What is the difference between LEAD() and LAG()? Provide a real-world use case.
10. How does FIRST_VALUE() differ from MIN() when used as a window function?
11. What is the difference between OVER() without PARTITION BY and with PARTITION BY?

SQL Window Functions – Interview Questions

1. Given a sales table, find the top 3 sales per region using window functions.
2. Find the difference in salary between each employee and the previous hired employee.
3. For each order, show the running total of sales amount per customer
4. Assign quartiles to products based on their revenue using NTILE(4).
5. Get the first order date and last order date for each customer using window functions.
6. Show the second-highest salary in each department without using LIMIT or TOP.
7. Calculate moving average of sales over the last 3 months.
8. Rank customers by their total purchase amount and handle ties correctly.
9. Show orders where the current order amount is greater than the previous order amount for that customer.
10. Find cumulative revenue percentage contribution per product within each category.

SQL Window Functions

Type of SQL Window Function	Description	Functions
Ranking Functions	Assign a ranking or position to each row based on order within a partition.	ROW_NUMBER(), RANK(), DENSE_RANK(), NTILE(n)
Aggregate Functions	Perform aggregate calculations over a specified window of rows.	SUM(), AVG(), MIN(), MAX(), COUNT()
Value Functions	Extract specific values from the window (first, last, or nth value).	FIRST_VALUE(), LAST_VALUE(), NTH_VALUE()
Navigation Functions	Access data from previous or next rows relative to the current row.	LAG(), LEAD()

SQL Window Functions

◆ What Are SQL Window Functions?

SQL Window Functions are advanced functions used to perform:

calculations across a set of table rows that are somehow related to the current row.

- ✓ Unlike aggregate functions that return a single result for a group, window functions preserve each row and add new computed columns.
- ✓ They are essential when you want aggregate-like behavior while still retaining row-level detail.

◆ Key Characteristics

- Operate on a window of rows around the current row.
- Do not collapse rows like GROUP BY or aggregate functions.
- Can be combined with PARTITION BY and ORDER BY for powerful row-wise calculations.
- Only usable in the SELECT, ORDER BY, or HAVING clauses.

SQL Window Functions

SQL Window Function Syntax

SELECT

column1,

column2,

window_function(column3) OVER (

[PARTITION BY partition_column]

[ORDER BY order_column]

[ROWS BETWEEN ...]

) AS alias_name

FROM

table_name;

window_function(column)

- A window function like ROW_NUMBER(), RANK(), SUM(), AVG(), etc., applied to a column.

OVER()

- The keyword that defines the window (i.e., the set of rows) over which the function operates.

PARTITION BY partition_column (optional)

- Divides the dataset into partitions/groups. The function is then applied within each partition separately.

ORDER BY order_column (optional but often required)

- Orders the rows within each partition. Many functions (like ROW_NUMBER()) depend on this order.

ROWS BETWEEN ... (optional)

- Defines a sliding frame of rows for the function to operate on. Example:
- ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW means include all previous rows up to the current one.

Understanding ROWS BETWEEN in SQL

```
<function>() OVER (  
    PARTITION BY <col>  
    ORDER BY <col>  
    ROWS BETWEEN <frame_start> AND <frame_end>  
)
```

ROWS BETWEEN frame_start **AND** frame_end

Frame Clause	Meaning	Example Use Case
UNBOUNDED PRECEDING	From first row in partition	Start of cumulative sum
N PRECEDING	N rows before current	Moving averages
CURRENT ROW	Current row only	Row-specific calculation
N FOLLOWING	N rows after current	Future-looking metrics
UNBOUNDED FOLLOWING	Last row in partition	Full partition totals

Combination	Frame	Example
UNBOUNDED PRECEDING AND CURRENT ROW	From start → Current	Cumulative sum
UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING	Full partition	Total for all rows
N PRECEDING AND CURRENT ROW	Moving window (past-focused)	Rolling sums
CURRENT ROW AND N FOLLOWING	Moving window (future-focused)	Forecast-like sums
N PRECEDING AND N FOLLOWING	Centered window	3-point moving average
CURRENT ROW AND UNBOUNDED FOLLOWING	Current → End	Future cumulative

Month	Revenue
Jan	100
Feb	300
Mar	200
Apr	500
May	400
Jun	600
Jul	700

SQL Value Window Functions

Value Window Functions help us extract specific values from the ordered set:

- **FIRST_VALUE()** – returns the first value in the window.
- **LAST_VALUE()** – returns the last value in the window.
- **NTH_VALUE()** – returns the nth value in the window.

```
FUNCTION_NAME (expression)
OVER (
  PARTITION BY column_name
  ORDER BY column_name
  ROWS/RANGE BETWEEN frame_start AND frame_end
)
```

Function	What it Returns	Key Point
FIRST_VALUE()	First value in window	Depends on ORDER BY
LAST_VALUE()	Last value in window	Needs rows between clause to avoid wrong results
NTH_VALUE()	nth value in window	Requires position n and correct frame

SQL Ranking Window Functions

Function	Behavior with Ties	Skips Ranks?	Use Case
ROW_NUMBER()	Always unique numbers	No	Sequential numbering
RANK()	Same rank for ties	Yes	Competition ranking
DENSE_RANK()	Same rank for ties	No	Category ranking
NTILE(n)	Divides into n groups	N/A	Percentiles, quartiles

SQL Navigation Window Functions

Navigation functions are a special category of window functions that help you access data from previous or next rows relative to the current row—without writing complex self-joins.

The two most common navigation functions are:

- LAG() → Looks “backward” to access data from a previous row.
- LEAD() → Looks “forward” to access data from a following row.

```
LAG(column, offset, default) OVER (  
    PARTITION BY partition_column  
    ORDER BY order_column  
)
```

```
LEAD(column, offset, default) OVER (  
    PARTITION BY partition_column  
    ORDER BY order_column  
)
```

Parameters:

- column → Column from which to fetch value.
- offset → (Optional, default = 1) Number of rows before/after current row to look.
- default → (Optional) Value to return if no row exists at that offset.
- PARTITION BY → Resets calculation for each group.
- ORDER BY → Defines order of rows to look before/after.

SQL Window Functions – Interview Questions

1. What are window functions in SQL? How do they differ from aggregate functions?
2. Explain the difference between GROUP BY and OVER() clause.
3. What is the difference between PARTITION BY and ORDER BY in a window function?
4. What are frames (ROWS BETWEEN / RANGE BETWEEN)? How do they affect calculations?
5. Difference between ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW vs. RANGE BETWEEN?
6. What is the default frame specification if you don't mention ROWS?
7. Explain the difference between RANK(), DENSE_RANK(), and ROW_NUMBER(). Give examples.
8. How does NTILE(n) work? Where can it be applied?
9. What is the difference between LEAD() and LAG()? Provide a real-world use case.
10. How does FIRST_VALUE() differ from MIN() when used as a window function?
11. What is the difference between OVER() without PARTITION BY and with PARTITION BY?

SQL Window Functions – Interview Questions

1. Given a sales table, find the top 3 sales per region using window functions.
2. Find the difference in salary between each employee and the previous hired employee.
3. For each order, show the running total of sales amount per customer
4. Assign quartiles to products based on their revenue using NTILE(4).
5. Get the first order date and last order date for each customer using window functions.
6. Show the second-highest salary in each department without using LIMIT or TOP.
7. Calculate moving average of sales over the last 3 months.
8. Rank customers by their total purchase amount and handle ties correctly.
9. Show orders where the current order amount is greater than the previous order amount for that customer.
10. Find cumulative revenue percentage contribution per product within each category.

